

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №1

По дисциплине «Интеллектуальный анализ данных»

Тема: «РСА»

Выполнил:

Студент 4 курса

Группы ИИ-26

Пасевич К.Ю.

Проверил:

Никитин С.К.

Брест 2026

Цель работы: научиться применять метод PCA для осуществления визуализации данных.

Ход работы

Общее задание

1. Используя выборку по варианту, осуществить проецирование данных на плоскость первых двух и трех главных компонент (двумя способами: 1. вручную через использование `numpy.linalg.eig` для вычисления собственных значений и собственных векторов и 2. с помощью `sklearn.decomposition.PCA` для непосредственного применения метода PCA – два независимых варианта решения);
2. Выполнить визуализацию полученных главных компонент с использованием средств библиотеки `matplotlib`, обозначая экземпляры разных классов с использованием разных цветовых маркеров;
3. Используя собственные значения, рассчитанные на этапе 1, вычислить потери, связанные с преобразованием по методу PCA. Сделать выводы;
4. Оформить отчет по выполненной работе, загрузить исходный код и отчет в соответствующий репозиторий на github.

| 9 | heart+failure+clinical+records.zip | death event |

Код программы:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
from sklearn.decomposition import PCA

DATA_PATH = "heart_failure_clinical_records_dataset.csv"
TARGET_COLUMN = "DEATH_EVENT"

def prepare_data(path: str = DATA_PATH):
    df = pd.read_csv(path)
    df.columns = df.columns.str.strip()

    target_col = None
    for col in df.columns:
        if col.lower() == TARGET_COLUMN.lower():
            target_col = col
```

```

        break

    if target_col is None:
        raise ValueError(
            f"Column '{TARGET_COLUMN}' not found. "
            f"Available columns: {list(df.columns)}"
        )

    feature_cols = [c for c in df.columns if c != target_col]

    df[feature_cols] = df[feature_cols].fillna(
        df[feature_cols].mean(numeric_only=True)
    )

    target = df[target_col].to_numpy()
    data = df[feature_cols].to_numpy(dtype=float)

    data = (data - data.mean(axis=0)) / data.std(axis=0, ddof=1)

    return data, target


def pca_manual(data: np.ndarray, n_components: int):
    data_centered = data - data.mean(axis=0)

    cov_matrix = np.cov(data_centered.T)
    eigenvalues, eigenvectors = np.linalg.eig(cov_matrix)

    sort_index = np.argsort(-1 * eigenvalues)
    eigenvalues_sorted = eigenvalues[sort_index]
    eigenvectors_sorted = eigenvectors[:, sort_index]

    principal_components = eigenvectors_sorted[:, :n_components]
    projected = np.dot(data_centered, principal_components)

    return projected.real, eigenvalues_sorted.real

```

```

def explained_loss(eigenvalues_sorted: np.ndarray, n_components: int) -> float:
    full_info = eigenvalues_sorted.sum()
    reduced_info = eigenvalues_sorted[:n_components].sum()
    loss_percent = 100 - reduced_info / full_info * 100
    return loss_percent


def pca_sklearn(data: np.ndarray, n_components: int):
    pca = PCA(n_components=n_components)
    projected = pca.fit_transform(data)
    return projected, pca.explained_variance_ratio_


def get_class_color(cls):
    colors = {0: "green", 1: "red"}
    return colors.get(int(cls), "gray")


def get_class_label(cls):
    labels = {0: "DEATH_EVENT = 0 (survived)", 1: "DEATH_EVENT = 1 (died)"}
    return labels.get(int(cls), f"class {int(cls)}")


def plot_2d(ax, projected_2d, target, title):
    for cls in np.unique(target):
        mask = target == cls
        ax.scatter(
            projected_2d[mask, 0],
            projected_2d[mask, 1],
            c=get_class_color(cls),
            label=get_class_label(cls),
            s=25,
        )
    ax.set_xlabel("PC1")
    ax.set_ylabel("PC2")
    ax.set_title(title)

```

```
ax.legend()
```

```
def plot_3d(ax, projected_3d, target, title):
```

```
    for cls in np.unique(target):
```

```
        mask = target == cls
```

```
        ax.scatter(
```

```
            projected_3d[mask, 0],
```

```
            projected_3d[mask, 1],
```

```
            projected_3d[mask, 2],
```

```
            c=get_class_color(cls),
```

```
            label=get_class_label(cls),
```

```
            s=25,
```

```
        )
```

```
ax.set_xlabel("PC1")
```

```
ax.set_ylabel("PC2")
```

```
ax.set_zlabel("PC3")
```

```
ax.set_title(title)
```

```
ax.legend()
```

```
def main():
```

```
    data, target = prepare_data()
```

```
    proj_manual_2d, eigvals = pca_manual(data, n_components=2)
```

```
    proj_manual_3d, _ = pca_manual(data, n_components=3)
```

```
    loss_manual_2d = explained_loss(eigvals, 2)
```

```
    loss_manual_3d = explained_loss(eigvals, 3)
```

```
    proj_sklearn_2d, var_ratio_2d = pca_sklearn(data, n_components=2)
```

```
    proj_sklearn_3d, var_ratio_3d = pca_sklearn(data, n_components=3)
```

```
    loss_sklearn_2d = 100 - var_ratio_2d.sum() * 100
```

```
    loss_sklearn_3d = 100 - var_ratio_3d.sum() * 100
```

```
    print(f"Loss (manual, 2 components): {loss_manual_2d:.2f}%")
```

```
    print(f"Loss (manual, 3 components): {loss_manual_3d:.2f}%")
```

```
    print(f"Loss (sklearn, 2 components): {loss_sklearn_2d:.2f}%")
```

```

print(f"Loss (sklearn, 3 components): {loss_sklearn_3d:.2f}%")

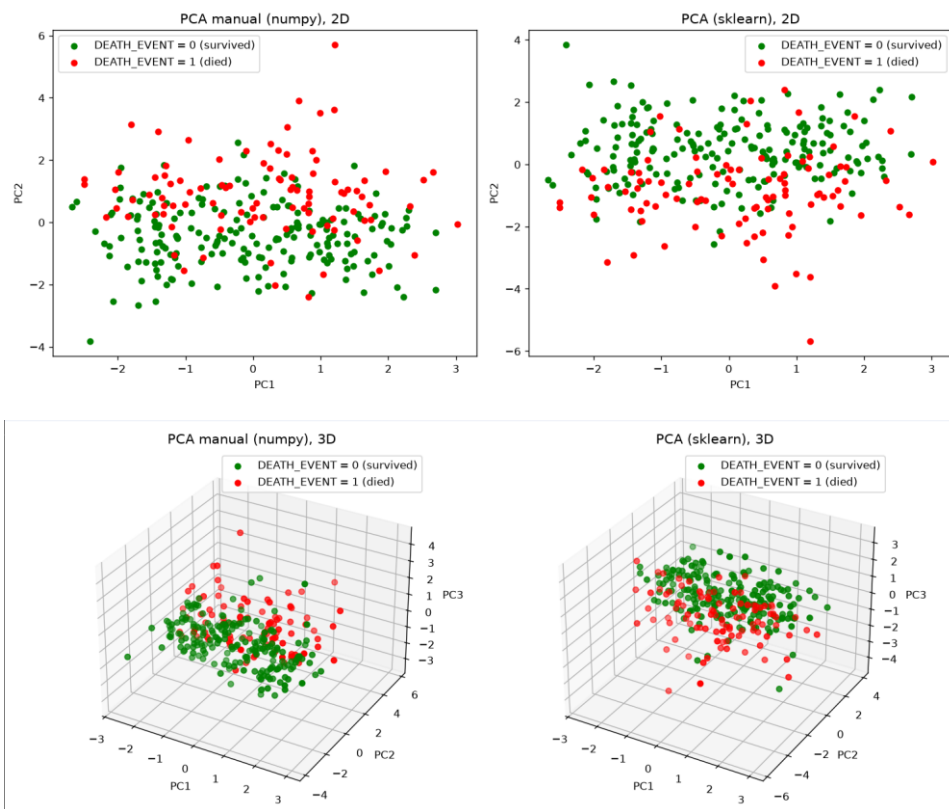
fig1, axes1 = plt.subplots(1, 2, figsize=(12, 5))
plot_2d(axes1[0], proj_manual_2d, target, "PCA manual (numpy), 2D")
plot_2d(axes1[1], proj_sklearn_2d, target, "PCA (sklearn), 2D")
fig1.tight_layout()
fig1.savefig("pca_2d.png", dpi=150)

fig2 = plt.figure(figsize=(12, 5))
ax_manual = fig2.add_subplot(1, 2, 1, projection="3d")
ax_sklearn = fig2.add_subplot(1, 2, 2, projection="3d")
plot_3d(ax_manual, proj_manual_3d, target, "PCA manual (numpy), 3D")
plot_3d(ax_sklearn, proj_sklearn_3d, target, "PCA (sklearn), 3D")
fig2.tight_layout()
fig2.savefig("pca_3d.png", dpi=150)
plt.show()

if __name__ == "__main__":
    main()

```

Результат программы:



Loss (manual, 2 components): 72.98%

Loss (manual, 3 components): 62.41%

Loss (sklearn, 2 components): 72.98%

Loss (sklearn, 3 components): 62.41%

В результате выполнения работы метод главных компонент был реализован двумя способами — вручную через `numpy.linalg.eig` и через `sklearn.decomposition.PCA`, — и оба дали полностью совпадающие результаты: потери информации составили 72.98% при двух компонентах и 62.41% при трёх. Это подтверждает корректность ручной реализации и одновременно показывает, что две-три компоненты сохраняют лишь около 27% и 38% исходной дисперсии. Такой высокий уровень потерь говорит о слабой коррелированности признаков датасета, а сильная перемешанность классов на графиках — о нелинейном характере связи между признаками и целевой переменной. Из этого следует, что метод главных компонент в данном случае применим прежде всего для снижения размерности и визуализации, но не для разделения классов.

Вывод: научилась применять метод PCA для осуществления визуализации данных.